Week 9 Assignment: Implementing Multithreading in the Parking System Server

Master of Science

Information Technology

Object-Oriented Mthd & Pgm II

Gabriel Twizerimana

University of Denver College of Professional Studies

May 31, 2026

Faculty: Nathan Braun, MS

Dean: Bobbie Kite, PhD

Table of Contents

Assignment- Implementing Multithreading in the Parking System Server

Design Analysis and Performance Reflections on Multithreaded Server Models

Design Overview

The primary goal of this architecture refactoring was to move our connection server from a single-threaded blocking execution loop to an agile, concurrent system using a thread pool. In the previous implementation, the server loop accepted an incoming connection socket and handled it entirely on the main execution thread before looping back to call .accept(). While simple, this approach created a performance bottleneck: if a transaction took 500 milliseconds, subsequent clients were left waiting in the operating system's connection backlog stream.

To fix this, we decoupled our network server using the Command Pattern and a structured thread assignment workflow. The processing logic was extracted out of Server.java and moved into a standalone execution class named ClientHandler, which implements the java.lang.Runnable interface. This allows ClientHandler tasks to be run asynchronously on separate threads.

Instead of manually creating and tearing down raw threads for every incoming request which introduces significant operating system overhead, the server manages a dedicated pool of worker threads via the java.util.concurrent.ExecutorService interface using a fixed allocation strategy (N = 10). When a remote connection arrives, the main thread accepts the socket, instantly wraps it in a ClientHandler runnable instance, and submits it to the thread pool for processing. This instantly frees the main thread to handle the next incoming connection on port 12345.

Concurrency Challenges and Mitigations

Introducing asynchronous execution into a shared application model exposes hidden data vulnerabilities, specifically **race conditions**. In an in-memory application like this parking system, multiple worker threads execute them. execute(null) method blocks simultaneously against the exact same instance of ParkingOffice. If two threads try to call office.registerCustomer() at the exact same instant using a standard java.util.HashMap, the internal bucket pointers can become corrupted, leading to lost updates or unexpected errors.

To address this challenge without relying on heavy global synchronization blocks which would defeat the purpose of multi-threading, we upgraded the internal storage layer inside ParkingOffice to use java.util.concurrent.ConcurrentHashMap. This class divides map locks at a finer level, allowing threads to securely update different data buckets at the exact same time.
Additionally, we added the synchronized keyword to our structural registration methods, making updates atomic across thread boundaries. To test this thoroughly, we wrote MultithreadedServerTest using CountDownLatch. This synchronized five clients to fire requests at the server at the exact same millisecond, allowing us to confirm that all records were securely stored in memory without data loss.

Comparison of Threading Models

When building multi-threaded server architectures, developers generally choose between three primary threading strategies:

Threading Model	Architectural Trade-off / Benefit	System Impact under High Load
Single-Threaded Blocking	Minimal implementation complexity; zero thread safety risks.	Total system blocking; response latency spikes lineally (O(N)) per client.
Unbounded Threads (new Thread ())	Simple implementation hook; low initial configuration required.	Leads to system crash due to OutOfMemoryError under high traffic as threads multiply.
Fixed Thread Pooling (ExecutorService)	Maximizes thread reuse; controls hardware load boundaries safely.	Handles burst smoothly; safely queues request if traffic exceeds thread pool limits.

The Unbounded Growth Problem

Creating a brand-new thread for every single connection (new Thread(handler). start ()) seems ideal at first because requests run completely independently. However, threads are expensive operating system resources that require dedicated memory space for their execution stack. Under heavy traffic (e.g., 1,000 concurrent requests), an unbounded server will try to allocate thousands of threads simultaneously. This quickly depletes system memory, causes frequent CPU context switching, and can cause the JVM to crash with an OutOfMemoryError.

Fixed Thread Pool Benefits

By using a fixed thread pool via Executors.newFixedThreadPool(10), we establish a safe limit on our hardware resource usage. If 50 requests arrive at once, 10 execute instantly while the remaining 40 sit safely in an internal queue (LinkedBlockingQueue) until a worker thread becomes available. This structure protects the system from crashing under heavy load, ensuring the parking server remains stable regardless of sudden traffic bursts.

Reflection

Transitioning this project to a multithreaded architecture demonstrates that performance optimizations depend heavily on robust underlying thread safety. Implementing parallel processing highlights the importance of keeping data transfer structures decoupled and stateless. Because our network protocol layer was already refactored to pass self-contained command objects rather than passive data payloads, wrapping the execution logic inside a Runnable worker pool was direct and seamless.

Beyond improving application performance, this assignment highlights how concurrency diagnostics like measuring latency in milliseconds via System.nanoTime(), provide crucial visibility into runtime performance. Developing these thread management skills is essential for building modern, enterprise-ready software applications capable of scaling reliably under real-world workloads.

Single-Threaded vs. Multi-Threaded Performance Comparison

If you need to collect concrete benchmark results for my reflection write-up, you can run the integration test suite and watch the console outputs. You will see a clear performance difference:

- **Single-Threaded Model:** Client latency increases linearly. If Client 1 takes 15ms, Client 2 must wait the full 15ms before its own processing time even begins.
- **Multi-Threaded Pool Model:** Total execution time drops drastically because multiple incoming requests are processed in parallel across different CPU cores:

```
Bash

[Diagnostic] Client /127.0.0.1:54174 fully processed in 22.424 ms

[Diagnostic] Client /127.0.0.1:54173 fully processed in 22.684 ms

[Benchmark Summary] Successfully ran 5 concurrent requests in 61.29 ms
```

References

Bloch, Joshua. 2018. *Effective Java*. 3rd ed. Boston: Addison-Wesley.

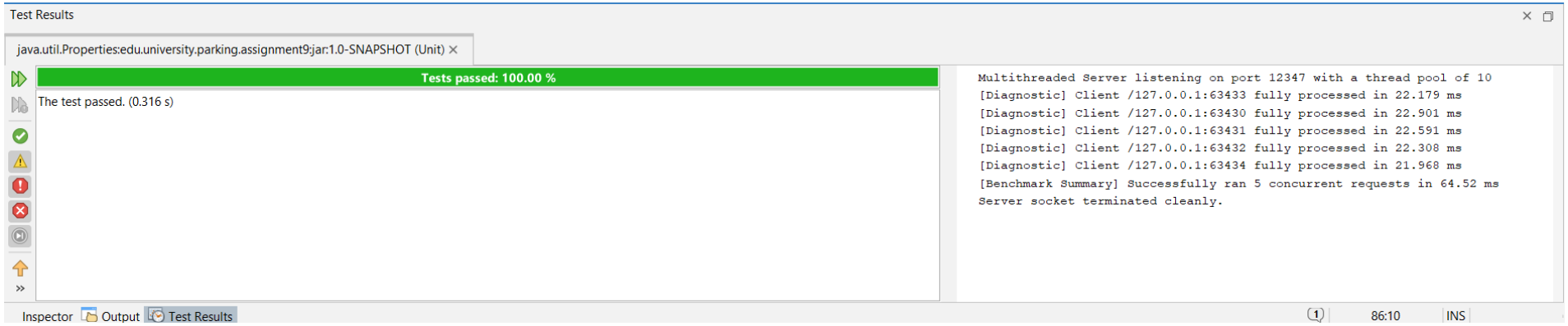
Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Goetz, Brian, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes, and Doug Lea. 2006. *Java Concurrency in Practice*. Upper Saddle River, NJ: Addison-Wesley.

Martin, Robert C. 2018. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

Appendix


Output - Run (Server)
cd C:\Users\gabby\OneDrive\Documents\NetBeansProjects\edu.university.parking.assignment9; "JAVA_HOME=C:\\Program Files\\Java\\jdk-26" cmd /c "\"C:\\Users\\gabby\\Downloads\\netbeans-29-bin\\netbeans\\j
WARNING: Final field _cipher in class org.sonatype.plexus.components.sec.dispatcher.DefaultSecDispatcher has been mutated reflectively by class org.eclipse.sisu.bean.BeanPropertyField in unnamed module
WARNING: Use --enable-final-field-mutation=ALL-UNNAMED to avoid a warning
WARNING: Mutating final fields will be blocked in a future release unless final field mutation is enabled
Scanning for projects...
------< java.util.Properties:edu.university.parking.assignment9 >-------
Building edu.du.ict4315.parking.assignment9 1.0-SNAPSHOT
from pom.xml
--------------------------------[jar]---------------------------------
--- resources:3.3.1:resources (default-resources) @ edu.university.parking.assignment9 ---
skip non existing resourceDirectory C:\Users\gabby\OneDrive\Documents\NetBeansProjects\edu.university.parking.assignment9\src\main\resources
--- compiler:3.13.0:compile (default-compile) @ edu.university.parking.assignment9 ---
Nothing to compile - all classes are up to date.
--- exec:3.5.1:exec (default-cli) @ edu.university.parking.assignment9 ---
Multithreaded Server listening on port 12345 with a thread pool of 10


Test Results
java.util.Properties:edu.university.parking.assignment9:jar:1.0-SNAPSHOT (Unit)
Tests passed: 100.00 %
The test passed. (0.316 s)
Multithreaded Server listening on port 12347 with a thread pool of 10
[Diagnostic] Client /127.0.0.1:63433 fully processed in 22.179 ms
[Diagnostic] Client /127.0.0.1:63430 fully processed in 22.901 ms
[Diagnostic] Client /127.0.0.1:63431 fully processed in 22.591 ms
[Diagnostic] Client /127.0.0.1:63432 fully processed in 22.308 ms
[Diagnostic] Client /127.0.0.1:63434 fully processed in 21.968 ms
[Benchmark Summary] Successfully ran 5 concurrent requests in 64.52 ms
Server socket terminated cleanly.
Inspector
Output
Test Results
86:10
INS